

STACKLY GYM & FITNESS STUDIO TECHNICAL DOCUMENTATION

Static website and dashboard web application — structure, styling, behaviour, responsiveness and deployment.

Version	1.0
Status	Released
Date	September 2026
Prepared by	Stackly Web Development Team
Subject area	Front-end / Static site & app UI

This document describes the file layout, design tokens, page inventory, responsive behaviour and the GitHub-based workflow used for the project. It is intended as a working reference for developers who take over or extend the codebase.

Contents

1. Introduction	4
1.1 Purpose of this document	4
1.2 Project overview	4
1.3 Scope	4
1.4 Reading conventions	4
2. Technology Stack	5
2.1 Core languages	5
2.2 Typography	5
2.3 External resources	5
3. Colour Palette	6
3.1 Brand colours	6
3.2 Extended greyscale and alpha scale	6
3.3 Where the palette is applied	7
3.4 Contrast and readability notes	7
4. Project Structure	7
4.1 Folder layout	7
4.2 Asset inventory	8
4.3 Shared components	8
5. Page Descriptions	10
5.1 index.html — Home	10
5.2 about.html — About	10
5.3 programs.html — Programs	10
5.4 coaches.html — Coaches	11
5.5 membership.html — Membership	11
5.6 contact.html — Contact	11
5.7 login.html and signup.html — Authentication	11
5.8 dashboard.html — Member dashboard	12
5.9 coach.html — Coach dashboard	12
5.10 404.html — Error page	12
6. Responsiveness	13
6.1 Breakpoint map	13
6.2 Behaviour notes	13

7. Deployment — GitHub 14

7.1 Repository setup 14

7.2 Daily workflow 14

7.3 Branching and release tags 14

7.4 Reverting changes 15

8. Conclusion 15

Appendix A — File inventory 15

Appendix B — Glossary 17

Page numbers in this contents list refer to the printed page numbers of this document.

1 Introduction

1.1 Purpose of this document

This document is a working reference for the Stackly Gym & Fitness Studio web project. It records what the project is, how the source is organised, what each page does, how the design system is built and how the project is managed in Git/GitHub. The document is written for anyone who needs to maintain, extend or hand over the codebase — a new developer joining the team, a reviewer, or the client's own technical staff.

It is not a user manual and it is not a visual design specification. It sits in between the two: it explains the structure and the decisions behind it in plain terms, with enough detail to let someone pick up the project without having to re-read every line of source.

1.2 Project overview

Stackly is a fitness studio brand based in Salem, Tamil Nadu. The web project consists of two things:

- **A marketing website** — a set of brochure-style pages (home, about, programs, coaches, membership, contact) plus two authentication pages (login, sign up) and an error page (404).
- **Two dashboard applications** — a member dashboard and a coach dashboard. These are single-page style interfaces, one file each, that simulate the day-to-day tools a member and a coach would use (bookings, schedules, progress, clients, programs, messaging and analytics).

The whole project is **static**: plain HTML files that link to stylesheet and script files. There is no server-side code, no database and no build step. All data shown in the dashboards is held in a single shared JavaScript data file (`data.js`) and rendered into the page at run time. Because there is no back end, every interactive feature works entirely in the browser.

1.3 Scope

In scope:

- All eleven HTML pages and their dedicated CSS and JavaScript files.
- The shared components: the mobile navigation bar and the brand logo.
- The colour system, typography and responsive rules.
- The GitHub-based version control workflow used for the project.

Out of scope:

- Back-end services, databases, payments or a live booking engine. These exist as simulated behaviour only.
- Hosting configuration beyond pushing the source to GitHub (the project is served as static files).
- A detailed style guide for marketing copy or brand voice.

1.4 Reading conventions

File names are written in this style throughout: `styles.css`. Class names and HTML attributes are written as they appear in source, for example `.stat-grid` or `data-route`. Code snippets use a monospaced font and are set apart from the body text. Where a section of the document is descriptive rather than technical, it is written as ordinary prose.

The tables in Sections 3, 4 and 5 should be read as quick-reference lists. The surrounding paragraphs give the reasoning behind them.

2 Technology Stack

2.1 Core languages

The project uses three web-standard languages, all hand-written and dependency-free.

Language	Used for	Notes
HTML5	Page structure	One document per page. Semantic elements (<code>header</code> , <code>nav</code> , <code>main</code> , <code>section</code> , <code>aside</code> , <code>footer</code>) are used throughout.
CSS3	Presentation and layout	Custom properties (CSS variables) carry the design tokens. Layout uses CSS Grid and Flexbox. No framework.
JavaScript (ES6+)	Behaviour and data rendering	Vanilla JavaScript. No libraries or frameworks. Scripts are written as plain files loaded with <code>script</code> tags.

There is intentionally no framework (React, Vue and so on) and no build tool (no `package.json`, no bundler). This keeps the project easy to open, easy to host and easy to understand — each page can be tested simply by opening the HTML file in a browser.

2.2 Typography

Two typefaces are loaded from Google Fonts and used consistently across all pages.

Typeface	Weight range loaded	Role
Barlow Condensed	500, 600, 700, 800, 900	Display type — headings, large numerals, section titles. Always set in uppercase in the interface.
DM Sans	400, 500, 600, 700	Body text, labels, buttons, tables and forms.

Fallbacks are declared in every stylesheet: `Barlow Condensed`, `Impact`, `sans-serif` for display and `"DM Sans"`, `Arial`, `sans-serif` for body text. Headings use a very tight line-height (around 0.9) and negative letter-spacing, which gives the condensed face its stacked, poster-like look. Body copy is set at around 14–15px with comfortable line spacing.

2.3 External resources

The project depends on two external resources, both loaded over HTTPS:

- **Google Fonts** — the two typefaces listed above, referenced in the `head` of every page.
- **Unsplash image CDN** — all photographic imagery (gym shots, coach portraits, avatars, testimonial photos) is served from `images.unsplash.com`. Images are requested in WebP format and sized with URL parameters so that each file stays under 100 KB.

In addition to those, the project keeps one local asset: the Stackly logo, stored as a WebP file in `images/logo/` and applied through CSS masks. Because the logo is a single colour on transparent background, it is drawn with `mask` and recoloured by changing the element background — the same file is reused for every light or dark header in the project.

3 Colour Palette

All colours in the project are declared once as CSS custom properties in the `:root` block of each stylesheet. Every page uses the same three brand colours; the dashboard pages add an extended scale of semi-transparent greys for depth.

3.1 Brand colours

Swatch	Name	Variable	Hex	Use
	Stackly Black	<code>--black</code>	<code>#0A0A0A</code>	Primary background. Nearly black with a slight warm cast; used on all dark sections and dashboards.
	Stackly Red	<code>--red</code>	<code>#E24B4A</code>	Accent and brand colour. Used for highlights, active states, primary buttons, icons and emphasis text.
	White	<code>--white</code>	<code>#FFFFFF</code>	Text on dark backgrounds and the background of the light sections of the marketing pages.

The red and black are the identity of the brand. The red is used sparingly so it reads as an accent — it marks what is active, clickable or important, and it is the only saturated colour in the palette.

3.2 Extended greyscale and alpha scale

The dashboard pages define a scale of white-based and red-based rgba values. These give the dark interface its layering without adding new hues.

Variable	Value	Purpose
<code>--muted</code>	<code>rgba(255,255,255,.56)</code>	Secondary text.
<code>--soft</code>	<code>rgba(255,255,255,.72)</code>	Primary text of reduced emphasis, link-like labels.
<code>--faint</code>	<code>rgba(255,255,255,.34)</code>	Placeholders, footnotes, disabled hints.
<code>--line</code>	<code>rgba(255,255,255,.12)</code>	Border / hairline between panels and rows.
<code>--line-strong</code>	<code>rgba(255,255,255,.21)</code>	Stronger border for cards, inputs and hover states.
<code>--glass</code>	<code>rgba(255,255,255,.055)</code>	Card / panel fill on top of the black background.
<code>--red-soft</code>	<code>rgba(226,75,74,.14)</code>	Subtle red tint behind active chips and callouts.

The marketing pages define two extra tokens for their light sections: `--ink-line` (`rgba(10,10,10,.16)`) for hairlines on white, and `--light-line` (`rgba(255,255,255,.18)`) for hairlines on dark. The mobile navigation bar keeps its own mirror set under the `--sk-` prefix so that the component never depends on the page it is dropped into.

3.3 Where the palette is applied

In practice the palette is used in three distinct looks:

- **Dark marketing pages** — black background, white text, red accents. Used by the home page hero, stats, pricing, gallery and final call-to-action sections.
- **Light marketing sections** — white background with black text and the same red accent. Used by the home page about, programs and testimonials sections.
- **Dashboard interfaces** — black chrome (sidebar, top bar) with glassy panels, the white-alpha scale for hierarchy, and red reserved for active items, key numbers and primary actions.

The red also appears as full solid blocks in several signature elements — the featured workout card, the plan card, the progress hero and the insight banner — where it carries black text.

3.4 Contrast and readability notes

- White text on the near-black background (contrast roughly 16:1) is used for all primary copy on dark surfaces.
- Muted and faint greys are used only for secondary content, so reduced-contrast text is never the only carrier of meaning.
- Red is always paired with either white or black text at large sizes; small red text (such as labels) is kept for accents where the surrounding contrast carries the content.
- Where a red block carries text, that text is always the near-black (#0A0A0A) to keep the reading contrast high.

4 Project Structure

4.1 Folder layout

Assets are grouped into three folders: `css/`, `js/` and `images/`. All HTML pages live at the root, one per page, so each page can be opened or linked directly.

```
stackly-gym-fitness/
├── css/                # stylesheets (one per page + shared nav)
│   ├── styles.css      # home page
│   ├── nav.css         # mobile navbar component (shared)
│   ├── about.css
│   ├── programs.css
│   ├── coaches.css
│   ├── membership.css
│   ├── contact.css
│   ├── auth.css        # login + signup
│   ├── dashboard.css   # member dashboard
│   ├── coach.css       # coach dashboard
│   └── 404.css
├── js/                # scripts (one per page + shared)
│   ├── main.js         # home page interactions
│   ├── nav.js          # mobile navbar behaviour (shared)
│   ├── data.js         # shared data for both dashboards
│   ├── about.js
│   ├── programs.js
│   ├── coaches.js
│   ├── membership.js
│   └── contact.js
```

```

├── auth.js          # login + signup
├── dashboard.js     # member dashboard logic
├── coach.js         # coach dashboard logic
├── 404.js
├── images/
│   └── logo/
│       └── logo-stackly.webp # brand logo (single colour, used as a mask)
├── index.html
├── about.html
├── programs.html
├── coaches.html
├── membership.html
├── contact.html
├── login.html
├── signup.html
├── dashboard.html
├── coach.html
└── 404.html

```

4.2 Asset inventory

Each page loads one dedicated stylesheet and one dedicated script, plus the shared mobile navigation component and, on the two dashboards, the shared data file. The table below maps pages to their assets.

Page	Stylesheet(s)	Script(s)	Notes
index.html	nav.css, styles.css	main.js, nav.js	Home / landing page.
about.html	nav.css, about.css	about.js, nav.js	Studio story page.
programs.html	nav.css, programs.css	programs.js, nav.js	Program catalogue page.
coaches.html	nav.css, coaches.css	coaches.js, nav.js	Coach team page.
membership.html	nav.css, membership.css	membership.js, nav.js	Membership plans page.
contact.html	nav.css, contact.css	contact.js, nav.js	Contact and enquiry page.
login.html	nav.css, auth.css	auth.js, nav.js	Member sign-in.
signup.html	nav.css, auth.css	auth.js, nav.js	New member registration.
dashboard.html	dashboard.css	data.js, dashboard.js	Member dashboard (no marketing nav).
coach.html	coach.css	data.js, coach.js	Coach dashboard (no marketing nav).
404.html	nav.css, 404.css	404.js, nav.js	Not-found page.

4.3 Shared components

Two pieces of code are reused across the marketing pages and are worth describing once.

The mobile navigation bar (nav.css + nav.js)

Every marketing and auth page includes a fixed top bar (`.sk-nav`) with the logo and a hamburger button, and a full-screen overlay menu (`.sk-menu`). The menu contains the site links, a contact block and social icons.

`nav.js` wires it up: it opens and closes the menu, traps focus inside it, closes on Escape, hides the bar on scroll-down and highlights the link for the current page. The component is shown only below 1024px — on desktop it hides itself and the regular page header takes over.

The logo (`images/logo/logo-stackly.webp`)

The logo is a single-colour WebP. It is applied with a CSS mask (`-webkit-mask / mask`) so the brand mark can be recoloured simply by changing the element's `background` — white on dark headers, red on hover, and black on light surfaces. The same approach is used in the mobile nav, the marketing headers and both dashboards.

5 Page Descriptions

This section walks through each page of the project in turn: what the page is for, what sections it contains, and which scripts drive its behaviour. Pages are described in the order they appear in the site's navigation.

5.1 index.html — Home

The landing page is the first page a visitor sees and carries the strongest brand statements. It is a long, single-scroll page built from one section after another.

- **Hero** — full-height opening with the headline “Train like you mean it.”, a framed photograph, a proof block (rating) and two calls to action.
- **Marquee** — an infinite scrolling strip of brand phrases.
- **About teaser** — a two-column gallery-and-copy section introducing the studio.
- **Stats** — four animated counters (members, trainers, years, floor area).
- **Programs rail** — a horizontally scrollable strip of four program cards.
- **Trainers** — a grid of coach cards with photo hover captions.
- **Why Stackly** — three feature items beside a full-bleed image.
- **Pricing** — three membership cards with one featured plan.
- **Testimonials** — a three-slide quote carousel with prev/next controls.
- **Gallery** — a masonry arrangement of studio photographs.
- **Final call-to-action** — closing panel with steps and a button.

Behaviour lives in `main.js`: animated counters, the testimonial slider, scroll-reveal of sections, and the marquee. The page also hosts the mobile navigation component.

5.2 about.html — About

The about page tells the studio's story. It opens with a hero, then a timeline of milestones, followed by a set of values with photographs.

- **Hero** — headline and lede about why the room exists.
- **Story / timeline** — dated milestones with media figures.
- **Values** — three photo-and-copy rows.
- **Philosophy** — a statement section with a large image and caption.
- **Studio tour** — a six-card collage (weights, cardio, boxing, yoga, locker rooms, recovery).
- **People** — coach cards with roles.

`about.js` drives the scroll-reveal animations and any gallery details on the page. Styling is provided by `about.css`; the mobile nav is shared.

5.3 programs.html — Programs

The programs page presents the training catalogue. It is the most interactive of the marketing pages.

- **Hero** — intro headline.
- **Program cards** — a grid of nine programs, each with a photo, difficulty indicator, duration and a category tag.

- **Filtering** — buttons at the top filter the grid by category (All, Strength, HIIT, Boxing, Yoga, Mobility, Cardio).
- **Spotlight** — a featured program with a mask reveal.
- **Training paths** — three tabbed panels (Beginner, Intermediate, Advanced).
- **Transformation cards** — flip cards showing before / result stats.

`programs.js` handles the category filter, the path tabs and the card interactions. The card grid, filters and tabs are rendered from a data structure inside the script.

5.4 coaches.html — Coaches

The coaches page introduces the coaching team and helps a visitor pick a coach.

- **Hero portraits** — four coach photographs in a row.
- **Coach cards** — a grid of four coaches with specialty tags; filterable by specialty (Strength, Conditioning, Movement).
- **The Stackly method** — a statement block with image.
- **Find your match** — an interactive tool: choosing a goal (get stronger / build engine / move without limits) swaps the recommended coach and description.
- **Profile panels** — a detail panel per coach, switched by selecting a card.

`coaches.js` holds the coach data (photo, quote, stats), powers the specialty filter and the match tool, and swaps the profile panels.

5.5 membership.html — Membership

The membership page explains the plans and answers common questions.

- **Hero** — membership headline.
- **Experience steps** — four numbered steps (walk in, get a read, find your rhythm, keep showing up).
- **Perks + FAQ** — a two-column layout with an image and an accordion of frequently asked questions.

`membership.js` handles the FAQ accordion (open/close) and scroll reveals.

5.6 contact.html — Contact

The contact page gathers enquiries and gives the visitor the studio's details.

- **Hero** — contact headline with background image.
- **Enquiry form** — name, email, subject and message fields with a submit action.
- **Contact details** — address, phone, email and visiting hours.

`contact.js` validates the form and provides the “message sent” feedback; there is no server, so submission is simulated in the browser.

5.7 login.html and signup.html — Authentication

The two authentication pages share the same stylesheet (`auth.css`) and script (`auth.js`).

- **login.html** — sign-in form (email, password) with a link to the sign-up page.
- **signup.html** — registration form (name, email, password) with a link to the login page.

Both pages show a side visual panel with a photograph. `auth.js` provides client-side validation and toggling of password visibility. On success the scripts redirect the visitor to the appropriate dashboard — login leads to the member dashboard, sign-up to the same dashboard.

5.8 dashboard.html — Member dashboard

The member dashboard is the logged-in experience for a member. It is a single HTML file with several views (sections) that are switched client-side.

- **Overview** — four stat cards, a weekly activity chart, a goal-completion donut, an up-next class card and quick actions.
- **Workouts** — a filterable/searchable library of workout cards with favourite and complete actions.
- **Schedule** — a week calendar with booked/available classes and a week switcher.
- **Progress** — a progress hero, weight trend line chart, personal-record bars, a capability map and a before/after comparison slider.
- **Membership** — plan card, usage meter and invoices table.
- **Achievements** — badge grid and a leaderboard.
- **Settings** — profile form and preferences toggles.

`data.js` holds the member, workout, schedule and achievement data. `dashboard.js` renders that data, draws the charts as inline SVG, handles the calendar, the comparison slider, the drawer and the view switching. State (favourites, completions, settings) is kept in `localStorage`.

5.9 coach.html — Coach dashboard

The coach dashboard is the counterpart for a coach. It shares the same application frame (sidebar, top bar) and the same data file.

- **Overview** — four stat cards, an earnings chart, a session-split donut, an up-next session card and quick actions.
- **Clients** — a grid of client cards with status tags, plus a slide-in drawer for client details, notes and recent workouts.
- **Sessions** — a week calendar colour-coded by session type, with complete/reschedule/cancel actions and an availability panel.
- **Programs** — a program builder form and a library of saved templates.
- **Analytics** — retention, attendance, earnings and rating-distribution charts.
- **Messages** — a client thread list with a chat panel.
- **Reviews** — rating summary and a filterable review list with reply forms.
- **Settings** — coach profile, preferences and an availability schedule editor.

`coach.js` renders the client data, drives the drawer, calendar and charts, and stores coaching notes and programs in `localStorage`.

5.10 404.html — Error page

The 404 page is a simple, on-brand error page with a large message, a link back to the home page and the shared mobile navigation. `404.js` adds a small touch of behaviour (such as the animated error background) while `404.css` styles the page.

6 Responsiveness

The whole project is responsive. It uses CSS media queries and fluid values (viewport-based `clamp()` font sizes) so the same markup adapts from a 320px phone up to a wide desktop screen. No separate mobile site is maintained.

6.1 Breakpoint map

Breakpoint	Where defined	Typical changes
$\geq 1024\text{px}$	nav.css	Mobile navigation bar is hidden; the regular page header is shown. The two dashboard layouts are unaffected.
$\leq 1050\text{px}$ / 1024px / 950px	marketing stylesheets (styles, about, programs, coaches, membership, contact)	Header CTA hidden, grids collapse from two/three columns to one, hero and section layouts stack.
$\leq 1100\text{px}$ / 1180px	dashboard.css, coach.css	Top bar compacts (breadcrumb and search collapse), stat grid drops to two columns, panel grids stack.
$\leq 760\text{px}$	all stylesheets	Primary mobile layout: single column, sidebar becomes a slide-in drawer, mobile menu button appears, calendars reduce to a minimum width with internal scrolling.
$\leq 480\text{px}$ / 450px	all stylesheets	Small-phone adjustments: tighter padding, smaller stat numbers, form columns collapse to one.
$\leq 360\text{px}$ / 320px	all stylesheets	Very narrow screens: logo and buttons shrink further, minimal page margins.

Table 6.1 — Media query breakpoints used across the project.

6.2 Behaviour notes

- **Mobile navigation.** Below 1024px the shared mobile bar replaces the legacy header. The menu is a full-screen overlay with staggered link animation, focus trapping and scroll-locking while open.
- **Dashboards.** The fixed sidebar (254px) collapses into a drawer on small screens; the top bar keeps a compact logo, search icon and avatar. Grids that are four columns on desktop become two, then one, on narrow screens.
- **Calendars.** The schedule week view keeps a minimum width (930px desktop, 840px mobile) inside an `overflow-x: auto` wrapper, so the wide 7-day grid scrolls horizontally inside its panel instead of breaking the layout.
- **Charts.** All charts are inline SVG that stretch with their containers; tooltips are positioned relative to the chart wrapper so they never overflow the page.
- **Images.** Photographs are served in WebP and capped under 100 KB (see Section 2.3), which keeps the pages light on mobile data connections.
- **Horizontal overflow control.** Both the `html` and `body` elements clip horizontal overflow on every stylesheet, so a stray wide element can never produce a page-level sideways scrollbar.
- **Reduced motion.** Every stylesheet includes a `prefers-reduced-motion` block that removes animations and transitions for users who ask for less motion.

7 Deployment — GitHub

The project is a static site, so “deployment” in this context means getting the source into a GitHub repository and keeping it in sync as changes are made. Because there is no build step, the files in the repository are exactly the files that get served.

7.1 Repository setup

To put the project under version control for the first time:

1. Create an empty repository on GitHub (no README, to avoid a merge conflict).
2. Initialise Git in the project folder and make the first commit.
3. Add the remote and push the branch.

```
# inside the project folder
git init
git add .
git commit -m "Initial commit: Stackly website and dashboards"
git branch -M main
git remote add origin https://github.com/<user>/<repo>.git
git push -u origin main
```

The `.gitignore`, if one is added, should only exclude editor/OS files (for example `.DS_Store` or `.vscode/`) — there are no dependencies or build outputs to ignore.

7.2 Daily workflow

For a small team the recommended routine is: pull the latest, make a focused change, commit it with a clear message, then push.

```
git pull                # get the latest work from origin
# ... make edits ...
git add css/ js/ index.html # stage only the files that changed
git commit -m "Add filter to the programs page"
git push
```

Conventions that keep the history readable:

- Commit small units of work with a single clear purpose.
- Write commit messages in the imperative mood (“Fix”, “Add”, “Update”, “Remove”).
- Stage only the files relevant to the change rather than using `git add .` every time.
- Check what will be committed with `git status` and review the diff with `git diff` before committing.

7.3 Branching and release tags

The project follows a simple branching model. `main` is the stable branch that always reflects a working site.

- **Feature branches.** New work (a page, a section, a fix) is done on a branch named for the work, then merged back to `main`.
- **Releases.** When a set of changes is considered done, it is tagged with a version number.

```
git checkout -b feature/member-schedule # work on a feature
# ... commit the work ...
git checkout main
git pull
git merge feature/member-schedule
git tag -a v1.1 -m "Release 1.1"
git push origin main --tags
```

7.4 Reverting changes

If a pushed change turns out to be wrong, the last commit can be undone without rewriting shared history:

```
git revert HEAD # creates a new commit that undoes the last one
git push
```

For local, un-pushed mistakes, `git reset` can be used instead. Commits already shared with the team should be reverted rather than reset, so the history stays safe for everyone.

8 Conclusion

This project is intentionally small and self-contained. It uses three plain web languages, no frameworks and no build tooling, which keeps every page fast to load, easy to host and straightforward to hand over. The effort has gone into craft rather than tooling: a consistent design system carried by a small set of CSS variables, one reusable mobile navigation component, and two dashboard interfaces that simulate real member and coach workflows entirely in the browser.

The structure is predictable — one HTML file per page, one stylesheet and one script per page, shared pieces in `css/nav.css`, `js/nav.js` and `js/data.js`. A new developer can find any feature quickly by following the page's file names, and can extend the dashboards by editing the data file rather than the markup.

Responsiveness is handled with a small set of breakpoints and fluid type, so the same files serve phones, tablets and desktops. Version control follows a plain GitHub workflow that suits a one-or-two-person team. Overall the project meets its goal: a polished, maintainable front end for the Stackly brand with no moving parts that can break.

9 Appendix A — File inventory

The complete list of source files in the project at version 1.0.

Path	Type	Purpose
index.html	HTML	Home / landing page.
about.html	HTML	About / story page.
programs.html	HTML	Program catalogue.
coaches.html	HTML	Coach team page.
membership.html	HTML	Membership plans.
contact.html	HTML	Contact page.
login.html	HTML	Member sign-in.
signup.html	HTML	Member registration.
dashboard.html	HTML	Member dashboard.
coach.html	HTML	Coach dashboard.
404.html	HTML	Error page.
css/styles.css	CSS	Home page styles.
css/nav.css	CSS	Mobile navigation component styles.
css/about.css	CSS	About page styles.
css/programs.css	CSS	Programs page styles.
css/coaches.css	CSS	Coaches page styles.
css/membership.css	CSS	Membership page styles.
css/contact.css	CSS	Contact page styles.
css/auth.css	CSS	Login / sign-up page styles.
css/dashboard.css	CSS	Member dashboard styles.
css/coach.css	CSS	Coach dashboard styles.
css/404.css	CSS	Error page styles.
js/main.js	JavaScript	Home page interactions.
js/nav.js	JavaScript	Mobile navigation behaviour.
js/data.js	JavaScript	Shared data for both dashboards.
js/about.js	JavaScript	About page behaviour.
js/programs.js	JavaScript	Programs filtering and tabs.
js/coaches.js	JavaScript	Coach filter and match tool.
js/membership.js	JavaScript	FAQ accordion.

js/contact.js	JavaScript	Contact form handling.
js/auth.js	JavaScript	Login / sign-up validation.
js/dashboard.js	JavaScript	Member dashboard logic.
js/coach.js	JavaScript	Coach dashboard logic.
js/404.js	JavaScript	Error page behaviour.
images/logo/logo-stackly.webp	Image	Brand logo (WebP).

10 Appendix B — Glossary

Term	Meaning
CSS custom property	A named value defined once (for example <code>--red</code>) and reused across a stylesheet. Also called a CSS variable.
Design token	A single reusable value — a colour, a font, a spacing unit — that represents one design decision.
Display type	A typeface used for large headings rather than body copy; here Barlow Condensed.
Flexbox / Grid	Two CSS layout systems used to arrange elements in rows, columns and grid cells.
Breakpoint	A viewport width at which a media query changes the layout.
clamp()	A CSS function that sets a value between a minimum and a maximum, scaling with the viewport.
localStorage	A browser storage area that persists data on the user's device between visits.
Mask	A CSS technique that uses an image's transparency to cut a shape out of an element's background.
Drawer	A panel that slides in from the side of the screen, used for the dashboard navigation and client details.
CDN	Content delivery network — a distributed server network; here used for fonts and photographs.
WebP	A modern image format with good compression; the project's images are served in this format.
Static site	A website whose files are served as-is, with no server-side processing or database.